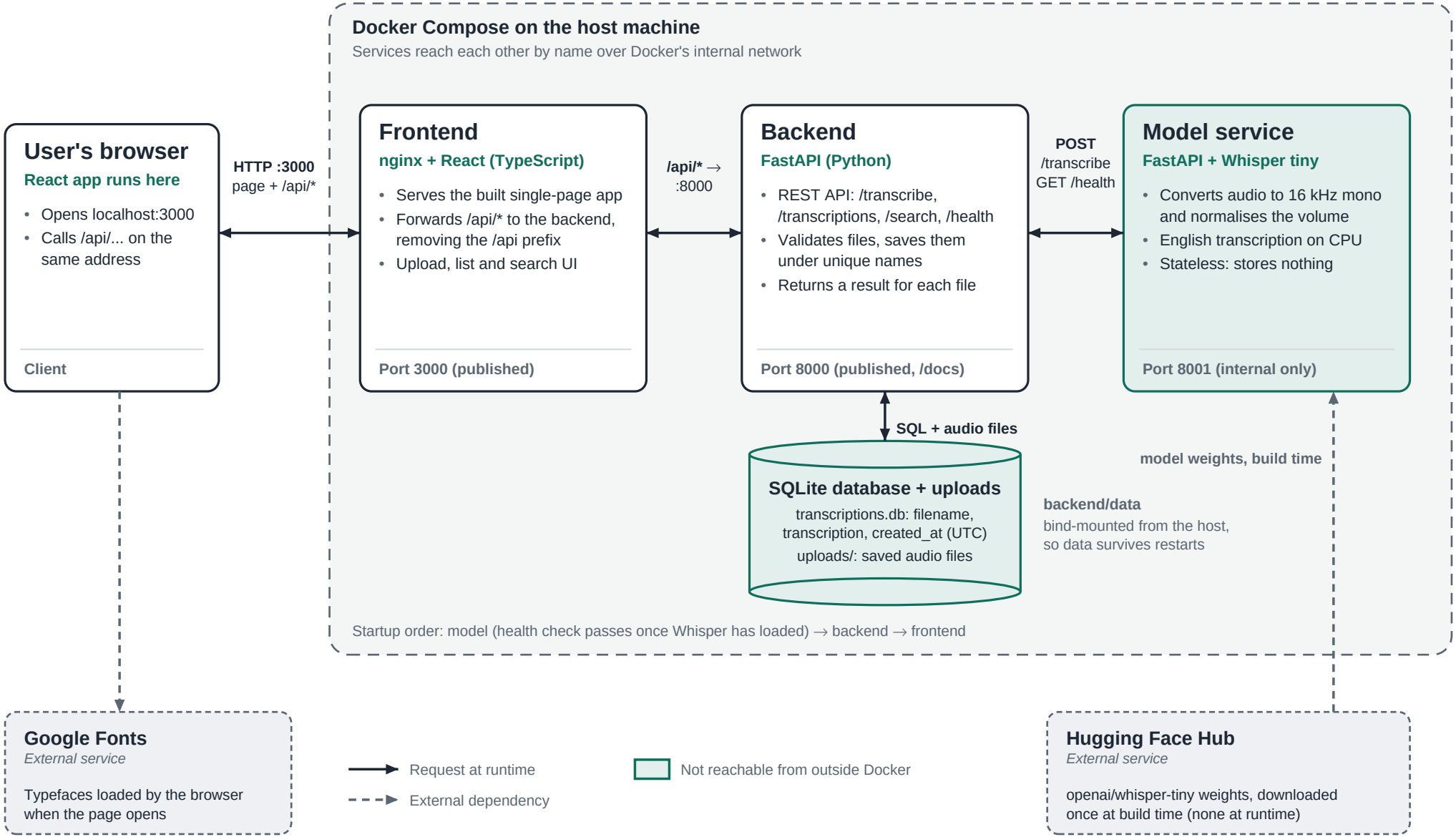
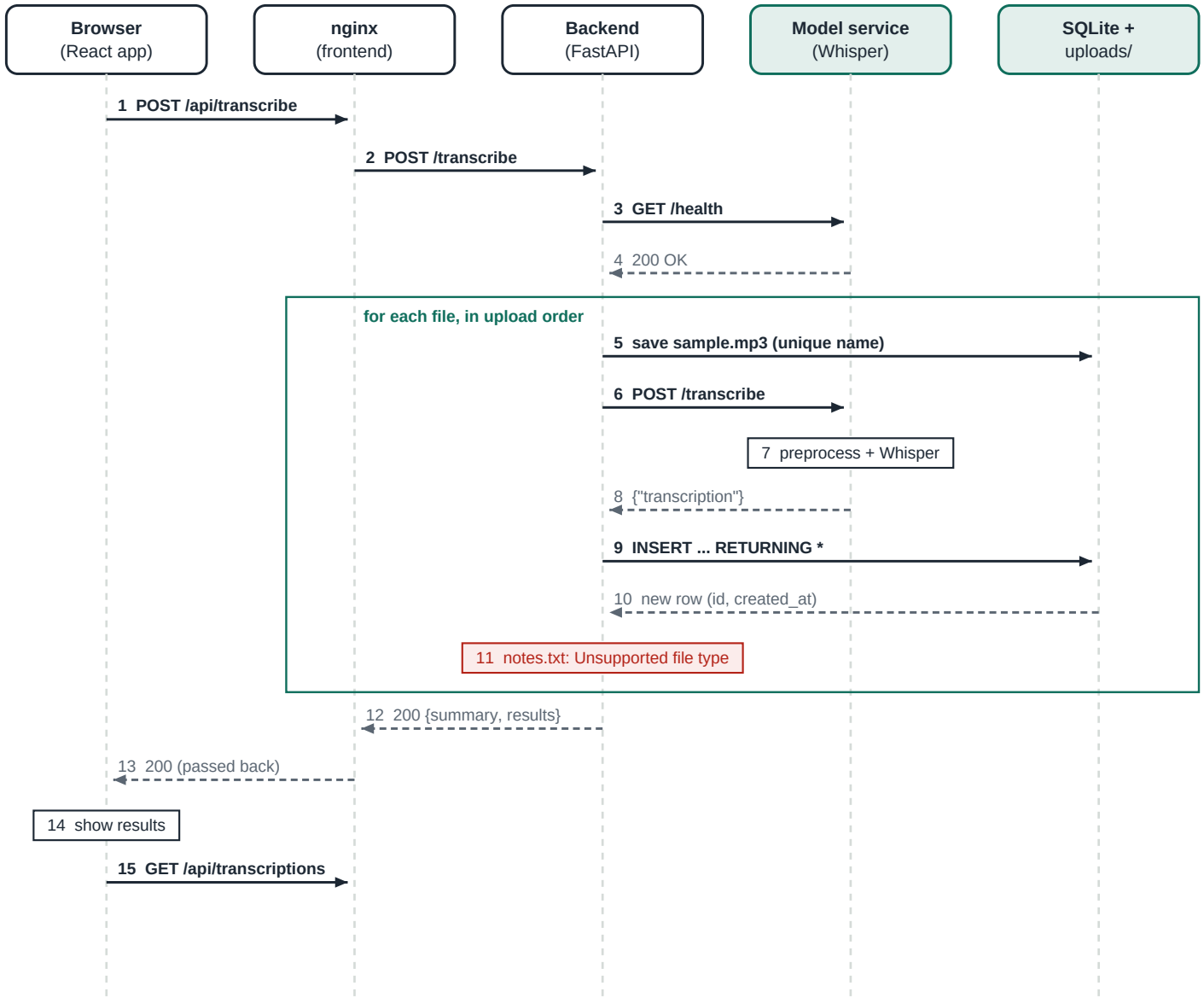
Audio Transcription: System Architecture

How the frontend, backend, model service, database and external services connect when the app runs with docker compose up.



1. Uploading audio, end to end

One request with two files: sample.mp3 is transcribed and saved, notes.txt is rejected.



How it works

- **Browser to backend:** the browser only calls localhost:3000; nginx removes /api and forwards the request over Docker's internal network (steps 1-2).
- **Health check first:** if the model service is down, the backend returns 503 before saving anything (steps 3-4).
- **One file at a time:** each file is saved under a unique name, transcribed, then stored with INSERT ... RETURNING * (steps 5-10).
- **Failures stay per file:** notes.txt is reported as an unsupported type while sample.mp3 is still saved (step 11).
- **One response:** the summary and per-file results return in a single 200 response, then the frontend reloads the list (steps 12-15).

Assumptions

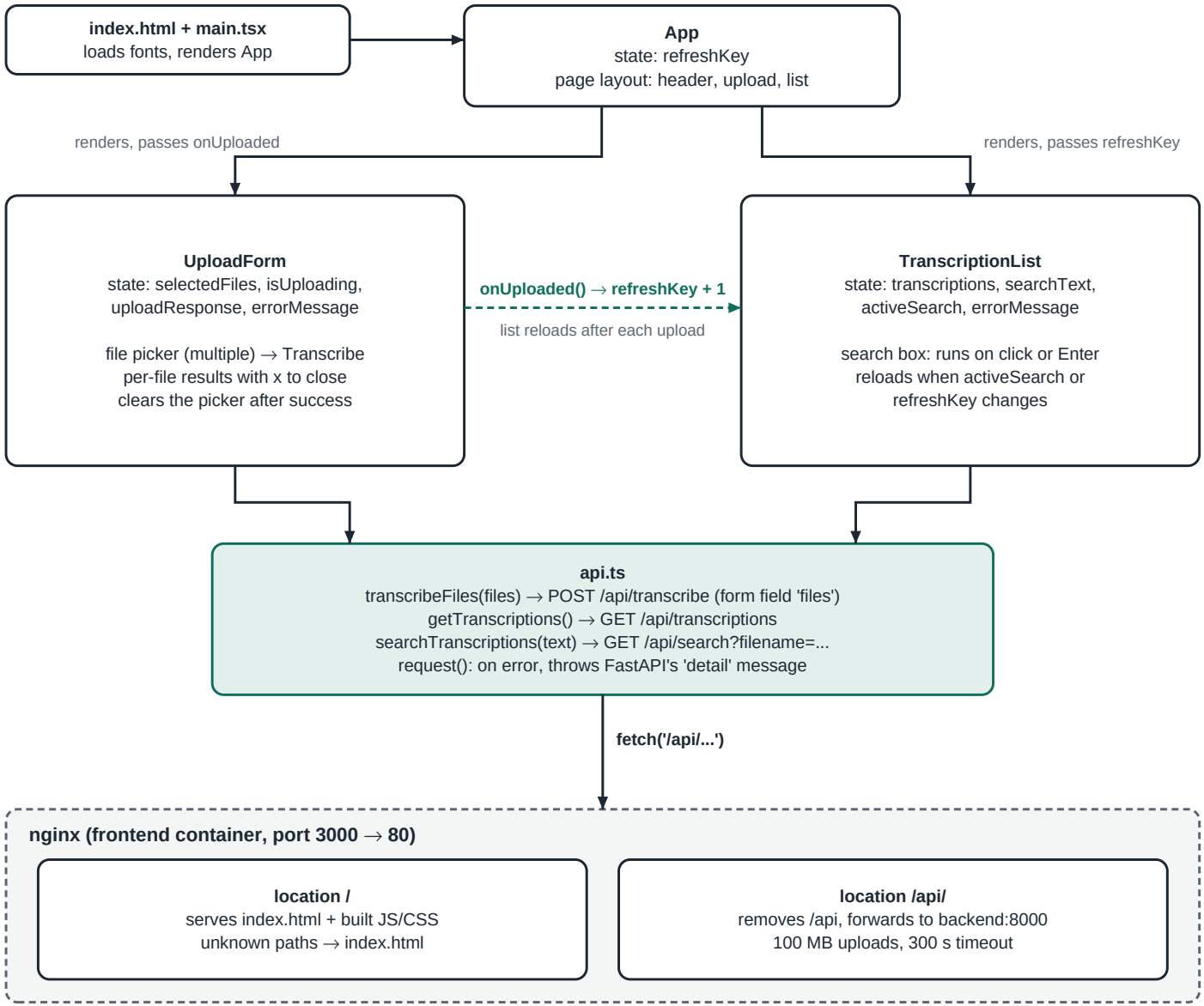
- **Files are reasonably short:** the backend waits up to 120 s per file and nginx up to 300 s per request.
- **Single user, low traffic:** files are processed one at a time, with no job queue.

Considerations

- **Startup order:** a health check starts the services in order (model, backend, frontend), so no upload arrives before Whisper has loaded.

2. Frontend

A React single-page app (TypeScript, Tailwind) for uploading audio, listing transcriptions and searching them.



How it works

- **Two feature components:** UploadForm handles single and batch uploads; TranscriptionList shows every transcription and the search results.
- **Linked through App:** after an upload, onUploaded() increases refreshKey and the list reloads.
- **Search on click:** typing only updates searchText; clicking Search runs one request, and responses from older searches are ignored.
- **One place for API calls:** api.ts holds every backend call and shows FastAPI's own error message.

Assumptions

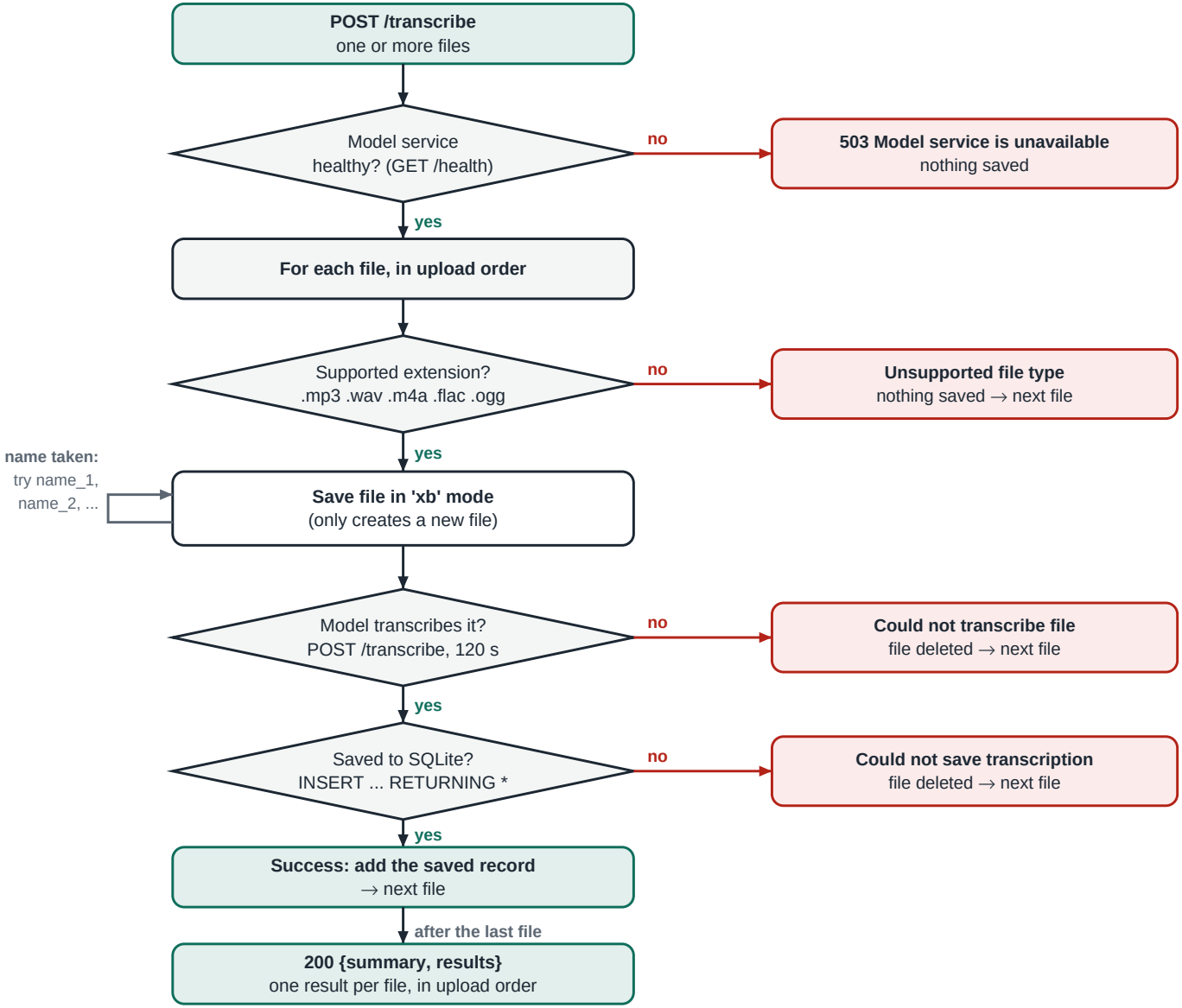
- **Modern browser, online:** the fonts load from Google Fonts when the page opens; without them the page uses system fonts.
- **Small number of transcriptions:** the full list loads at once, without pagination.

Considerations

- **One entry point:** API calls go through /api on the frontend's own address, which avoids cross-origin (CORS) restrictions without changing the backend.
- **nginx:** serves the built files and re-checks the backend's address every 10 s, so the app keeps working if the backend container is recreated.
- **Port 3000 → 80:** nginx listens on port 80 inside its container, and Docker publishes it as port 3000 on the host, so the app opens at localhost:3000.
- **Testing:** unit tests fake the backend and run in Docker.

3. Backend: processing an upload

A FastAPI service: the decisions POST /transcribe makes for a batch, and what happens when a step fails.



How it works

- **Fail fast:** one health check before the batch; if the model service is down, the response is 503 and nothing is saved.
- **Per-file results:** each file succeeds or fails on its own, so one bad file never discards the others; the response is 200 with a summary.
- **Unique names:** 'xb' mode only creates a file that doesn't exist yet, so two uploads with the same name can't overwrite each other.
- **No orphan files:** if transcription or saving fails, the upload is deleted, so every file on disk has a database row.

Assumptions

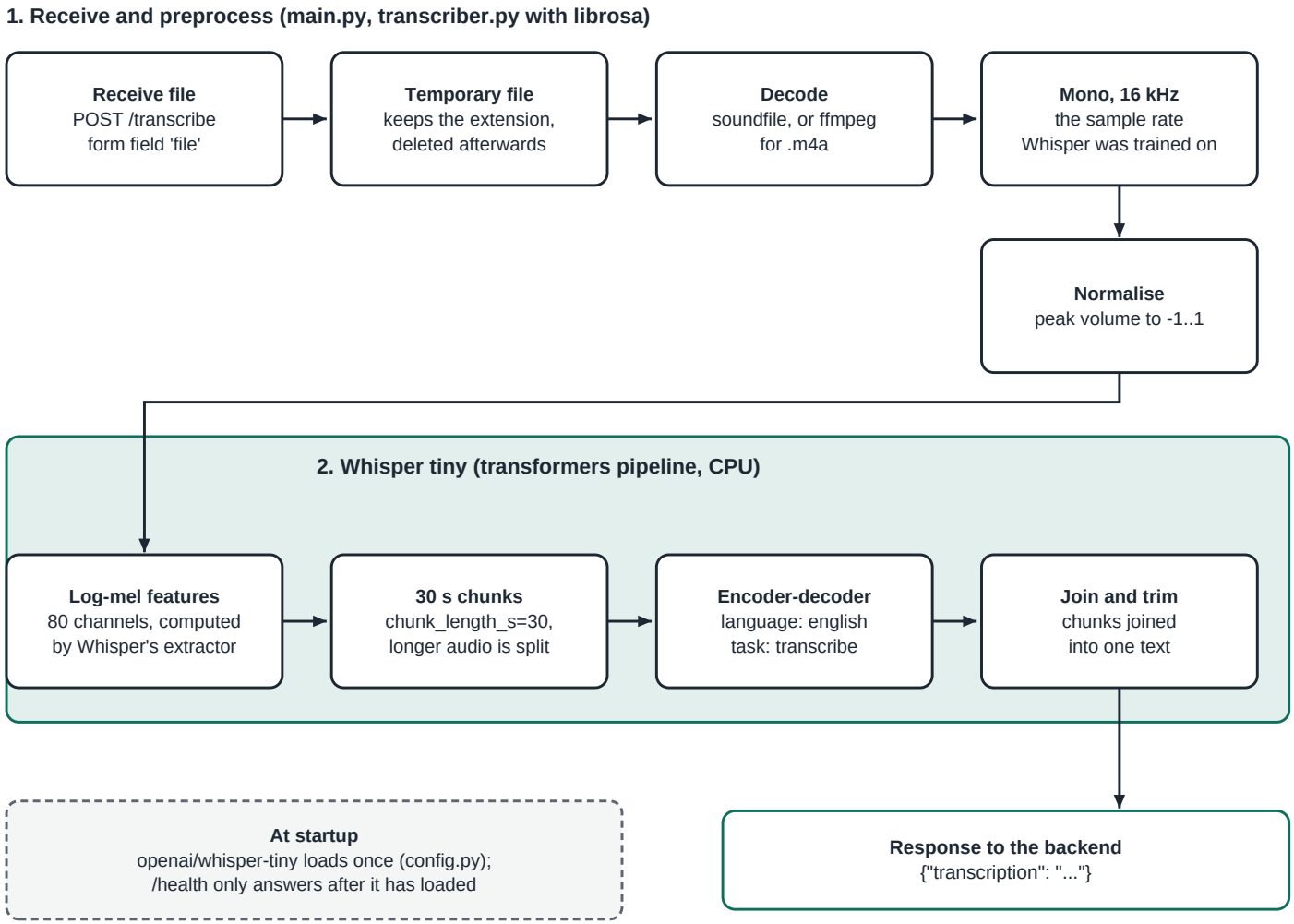
- **File type by extension:** .mp3, .wav, .m4a, .flac or .ogg; files that can't be decoded are reported per file.
- **Runs locally:** single user, so there is no authentication.

Considerations

- **Testing:** unit tests fake the model service, use a temporary data folder and run in Docker.

4. Model service: audio to text

A separate FastAPI service running Whisper tiny: the preprocessing and inference steps for one file.



How it works

- **Temporary file:** ffmpeg-based decoding (needed for .m4a) reads from a file path; the file is deleted once transcription ends.
- **16 kHz mono, normalised:** the format Whisper was trained on; quiet recordings work as well as loud ones.
- **Chunking:** audio longer than 30 s is split into chunks and the text joined.
- **Stateless:** it keeps nothing; the backend owns the files and database.

Assumptions

- **English audio:** Whisper tiny's language detection proved unreliable (English came out as Malay), so English is forced.
- **Separate model services:** mimics a setup like HTX's Ngine platform, where AI models are standalone services called through APIs.
- **Inference optimisation out of scope:** one file per request, on the CPU.

Considerations

- **Independent:** the model can be changed, scaled or moved to a GPU without touching the backend.
- **Security:** it decodes untrusted audio, so its port is not published outside Docker.
- **Lightweight and offline:** CPU-only PyTorch; weights come from the Hugging Face Hub at build time, not at runtime.

5. Database and file storage

The SQLite transcriptions table, how it maps to the stored files, and the queries behind each endpoint.

transcriptions (transcriptions.db)

id	INTEGER	primary key, auto-increment
filename	TEXT	NOT NULL, UNIQUE (stored name)
transcription	TEXT	NOT NULL
created_at	TEXT	default: UTC, e.g. 2026-10-14T08:41:00Z

same name

backend/data on the host
= /app/data in the container

transcriptions.db
uploads/
sample.mp3
sample_1.mp3

one row each, same filename

Queries (database.py), one connection per call, values passed as ? placeholders

POST /transcribe endpoint

```
INSERT INTO transcriptions (filename, transcription) VALUES (?, ?) RETURNING *
```

returns the new row (id, created_at) without a second query

GET /transcriptions endpoint

```
SELECT * FROM transcriptions ORDER BY id
```

all rows, oldest first

GET /search endpoint

```
SELECT * FROM transcriptions WHERE filename LIKE ? ESCAPE '\' ORDER BY id
```

parameter: %text%, with \, % and _ escaped so they match literally

On startup
CREATE TABLE IF NOT EXISTS creates the table the first time. The data folder comes from DATA_DIRECTORY (tests use a temporary folder).

How it works

- **One table:** the stored file name, the transcription and a UTC timestamp set by SQLite.
- **Files and rows match:** a file in uploads/ and its row share the same name; UNIQUE on filename keeps them one-to-one.
- **UTC with Z:** the frontend shows times in the viewer's own timezone.
- **Searching:** GET /api/search runs a partial, case-insensitive LIKE on the file names, with %, _ and \ escaped so they match literally.

Considerations

- **Safe queries:** values are passed as ? placeholders, never built into the SQL, which prevents SQL injection.
- **Persistence:** backend/data is bind-mounted from the host, so data survives restarts and is easy to inspect; a named volume would suit production.